

```

"""
Program 6: Implement Quick Sort and Merge
Sort and observe the
execution time for various input sizes
(Average, Worst and Best cases).

Best case (Quick Sort) -> already sorted
list, using a random-pivot
                                quicksort still
runs close to  $O(n \log n)$ ,
                                here we also
show a "sorted" array case.
Worst case (Quick Sort) -> sorted /
reverse-sorted array with a
                                naive last-
element pivot degrades to  $O(n^2)$ .
Average case                    -> randomly
shuffled array.

Merge Sort is always  $O(n \log n)$  regardless
of input arrangement,
so its timings are shown for comparison
across the same cases.
"""

import time
import random
import sys

sys.setrecursionlimit(10000)

```

```
def quick_sort(arr):
    # naive last-element pivot (used to
    demonstrate worst case on sorted input)
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left = [x for x in arr[:-1] if x <=
pivot]
    right = [x for x in arr[:-1] if x >
pivot]
    return quick_sort(left) + [pivot] +
quick_sort(right)

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
```

```

        j += 1
        result.extend(left[i:])
        result.extend(right[j:])
        return result

def time_it(sort_fn, arr):
    start = time.perf_counter()
    sort_fn(arr.copy())
    end = time.perf_counter()
    return end - start

def main():
    sizes = list(map(int, input("Enter
input sizes separated by space (e.g. 100
500 1000): ").split()))

    print(f"\n{'Size':>8} | {'Case':>10} |
{'QuickSort(s)':>14} |
{'MergeSort(s)':>14}")
    print("-" * 55)

    for n in sizes:
        best_case =
list(range(n))                # already
sorted (best-ish for quicksort's recursion
balance is actually average with random
pivot; naive last-pivot => this is worst)
        worst_case = list(range(n, 0,
-1))                # reverse sorted
        avg_case = list(range(n))
        random.shuffle(avg_case)

```

```

        for label, data in [("Sorted",
best_case), ("Reverse", worst_case),
("Random", avg_case)]:
            qt = time_it(quick_sort, data)
            mt = time_it(merge_sort, data)
            print(f"{n:>8} | {label:>10} |
{qt:>14.6f} | {mt:>14.6f}")

    print("\nNote: With this naive last-
element-pivot Quick Sort, 'Sorted' and")
    print("'Reverse' inputs are the WORST
case ( $O(n^2)$ ), while 'Random' input")
    print("shows the AVERAGE/BEST-like case
(close to  $O(n \log n)$ ).")
    print("Merge Sort stays close to  $O(n
\log n)$  for all cases.")

if __name__ == "__main__":
    main()

```